

链接实验操作指导

命令列表

- **gcc**
- **gdb**
- **readelf**
- **objdump**
- **objcopy**
- **hexdump**

gcc

`gcc -E test.c -o test.i` #预处理

`gcc -S test.i -o test.s` #编译汇编

`gcc -S -fverbose-asm test.c -o test.s` #编译汇编，含源码

`gcc -c -g test.c -o test.o` #生成可重定位的目标文件

`gcc -g test.c -o test` #链接生成可执行文件

`gcc -g -D U1 main.c sub.c -o main_sub -static` 静态链接

自帶的幫助

- **readelf --help**

Usage: **readelf** <option(s)> elf-file(s)

Options are:

-a --all	Equivalent to: -h -l -S -s -r -d -V -A -l
-h --file-header	Display the ELF file header
-l --program-headers	Display the program headers
--segments	An alias for --program-headers
-S --section-headers	Display the sections' header
--sections	An alias for --section-headers
-g --section-groups	Display the section groups
-t --section-details	Display the section details
-e --headers	Equivalent to: -h -l -S
-s --syms	Display the symbol table
--symbols	An alias for --syms

自帶的幫助

<code>--dyn-syms</code>	Display the dynamic symbol table
<code>-n --notes</code>	Display the core notes (if present)
<code>-r --relocs</code>	Display the relocations (if present)
<code>-u --unwind</code>	Display the unwind info (if present)
<code>-d --dynamic</code>	Display the dynamic section (if present)
<code>-V --version-info</code>	Display the version sections
<code>-A --arch-specific</code>	Display architecture specific information
<code>-c --archive-index</code>	Display the symbol/file index in an archive
<code>-D --use-dynamic</code>	Use the dynamic section info when displaying symbols
<code>-x --hex-dump=<number name></code>	
	Dump the contents of section <number name> as bytes
<code>-p --string-dump=<number name></code>	
	Dump the contents of section <number name> as strings
<code>-R --relocated-dump=<number name></code>	
	Dump contents of section <number name> as relocated bytes
<code>-z --decompress</code>	Decompress section before dumping it

自帶的幫助

`-w[lLiaprmfFsoRtUuTgAckK]` or
`--debugdump[=rawline,=decodedline,=info,=abbrev,=pubnames,
=aranges,=macro,=frames,=framesinterp,=str,=loc,=Ranges,
=pubtypes,=gdb_index,=trace_info,=trace_abbrev,
=trace_aranges,=addr,=cu_index,=links,=follow-links]`
Display the contents of DWARF debug sections
`--dwarf-depth=N` Do not display DIEs at depth N or greater
`--dwarf-start=N` Display DIEs starting with N, at the same
depth or deeper
`-I --histogram` Display histogram of bucket list lengths
`-W --wide` Allow output width to exceed 80 characters
`@<file>` Read options from <file>
`-H --help` Display this information
`-v --version` Display the version number of readelf

可重定位目标文件的数据结构

□ 可重定位目标文件的组成

`/usr/include/elf.h`

- 文件头 : ELF 头 参见结构 : `Elf64_Ehdr`
- 节 表 : 对各个节描述, 由描述项形成的表
参见结构 : `Elf64_Shdr`
- 节 : 节内的具体信息

显示文件头

❑ **struct : Elf64_Ehdr**

#readelf -h test.o

#readelf --file-header test.o

```
root@LAPTOP-CJLSTBTI:/home/chapter4# readelf -h test.o
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                                   2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  REL (Relocatable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x0
  Start of program headers:               0 (bytes into file)
  Start of section headers:               5672 (bytes into file)
  Flags:                                  0x0
  Size of this header:                     64 (bytes)
  Size of program headers:                 0 (bytes)
  Number of program headers:               0
  Size of section headers:                 64 (bytes)
  Number of section headers:               21
  Section header string table index:      20
```


显示文件头

readelf -h 显示文字	Elf64_Ehdr 字段名	字段含义
Magic	e_ident[16]	ELF 标识字节，包含魔数、64/32 位、大小端、ABI 等
Type	e_type	文件类型（REL 可重定位、EXEC 可执行、DYN 动态库）
Machine	e_machine	目标架构（x86-64、ARM、RISC-V 等）
Version	e_version	ELF 版本号，通常为 1
Entry point address	e_entry	程序入口虚拟地址（.o 文件一般为 0）
Start of program headers	e_phoff	程序头表在文件中的偏移
Start of section headers	e_shoff	节头表在文件中的偏移
Flags	e_flags	处理器相关标志
Size of this header	e_ehsize	ELF 文件头自身大小（64 位 ELF 固定 64 字节）
Size of program headers	e_phentsize	单个程序头表项的大小
Number of program headers	e_phnum	程序头表项总数
Size of section headers	e_shentsize	单个节头表项的大小（64 位通常 64 字节）
Number of section headers	e_shnum	节头表项总数
Section header string table index	e_shstrndx	节名所在字符串表的节索引

显示节头表

□ 节头表数据结构

Elf64_Shdr

- **readelf -S test.o**
- **readelf --section-headers test.o**
- **readelf --sections test.o**
- **readelf -S -W test.o** 以宽行的形式显示节头信息

There are 21 section headers, starting at offset 0x1628:

Section Headers:

[Nr]	Name	Type	Address	Off	Size	ES	Flg	Lk	Inf	Al
[0]		NULL	0000000000000000	000000	000000	00		0	0	0
[1]	.text	PROGBITS	0000000000000000	000040	0000e9	00	AX	0	0	1
[2]	.rela.text	RELA	0000000000000000	000d20	000180	18	I 18	1	8	
[3]	.data	PROGBITS	0000000000000000	00012c	000008	00	WA	0	0	4
[4]	.bss	NOBITS	0000000000000000	000134	000000	00	WA	0	0	1
[5]	.rodata	PROGBITS	0000000000000000	000138	000040	00	A	0	0	8
[6]	.debug_info	PROGBITS	0000000000000000	000178	0003bc	00		0	0	1
[7]	.rela.debug_info	RELA	0000000000000000	000ea0	000660	18	I 18	6	8	

显示节头表

adelf -S 显示列名	Elf64_Shdr 字段名	字段含义
Nr	(索引)	节的编号（非结构体字段，是顺序位置）
Name	sh_name	节名（在节名字符串表中的偏移）
Type	sh_type	节类型（如：PROGBITS, STRTAB, SYMTAB 等）
Address	sh_addr	加载到内存时的虚拟地址（.o 文件通常为 0）
Off	sh_offset	本节在文件中的字节偏移（定位数据的关键）
Size	sh_size	本节在文件 / 内存中的大小
ES	sh_entsize	表项大小（若节是表结构，如符号表每个条目大小）
Flg	sh_flags	节标志（W = 可写，A = 可加载，X = 可执行）
Lk	sh_link	节链接信息（依赖的其他节索引）
Inf	sh_info	附加信息（根据节类型不同而变化）
Al	sh_addralign	地址对齐要求（如 4, 8, 16 字节对齐）

显示某个节中的内容

□ 显示代码节中的内容 以 16进制字节形式显示

➤ `readelf -x .text test.o`

```
root@LAPTOP-CJLSTBTI:/home/chapter4# readelf -x .text test.o

Hex dump of section '.text':
NOTE: This section has relocations against it, but these have NOT
0x00000000 554889e5 897dec89 75e88b55 ec8b45e8 UH...}..u..U..E.
0x00000010 01d08945 fc8b1500 0000008b 05000000 ...E.....
0x00000020 0001d089 45fc8b15 00000000 8b050000 ....E.....
0x00000030 000001d0 8945fc8b 45fc5dc3 554889e5 .....E..E.].UH..
0x00000040 4883ec30 64488b04 25280000 00488945 H..0dH..%(...H.E
0x00000050 f831c048 8d45e048 be68656c 6c6f2c20 .1.H.E.H.hello,
0x00000060 31488930 c7400832 33343566 c7400c21 1H.0. @.2345f. @. !
0x00000070 00be1400 0000bf0a 000000e8 00000000 .....
0x00000080 8945dc48 8d0d0000 0000488d 15000000 .E.H.....H.....
```

`# readelf -x 1 test.o` `.text` 节的编号为 1

Hex dump of section '.text':

.....

显示某个节中的内容

□ 显示代码节中的内容 以 16进制字节形式显示

```
# objcopy -O binary --only-section=.text test.o  
/dev/stdout | hexdump -C
```

```
# objdump -d test.o
```

请比较两个命令执行结果

显示某个节中的内容

□ 显示数据节中的内容 以 16进制字节形式显示

➤ `readelf -x .data test.o`

```
root@LAPTOP-CJLSTBTI:/home/chapter4# readelf -x .data test.o
Hex dump of section '.data':
 0x00000000 14000000 1e000000                      . . . . .
```

说明：程序中有三个全局变量，但只有 2个初始化了。

```
int gx=20;
```

```
int gy;
```

```
int gz=30;
```

逐字节显示（字节与字节之间有空格）

`objcopy -O binary --only-section=.data test.o /dev/stdout | hexdump -C`

`objdump -s --section=.data main.o`

显示某个节中的内容

```
root@LAPTOP-CJLSTBTI:/home/chapter4# readelf -x .strtab test.o
```

```
Hex dump of section '.strtab':
```

```
0x00000000 00746573 742e6300 67780067 7900677a .test.c.gx.gy.gz
0x00000010 00616464 006d6169 6e005f47 4c4f4241 .add.main._GLOBA
0x00000020 4c5f4f46 46534554 5f544142 4c455f00 L_OFFSET_TABLE_
0x00000030 7072696e 7466005f 5f737461 636b5f63 printf.__stack_c
0x00000040 686b5f66 61696c00                hk_fail.
```

□ 显示字符串节中的内容 以字符串的形式显示

```
root@LAPTOP-CJLSTBTI:/home/chapter4# readelf -p .strtab test.o
```

```
String dump of section '.strtab':
```

```
[ 1] test.c
[ 8] gx
[ b] gy
[ e] gz
[11] add
[15] main
[1a] _GLOBAL_OFFSET_TABLE_
[30] printf
[37] __stack_chk_fail
```

显示某个节中的内容

□ 显示节头部字符串节中的内容 以 字符串的形式显示

```
root@LAPTOP-CJLSTBT1:/home/chapter4# readelf -p .shstrtab test.o
```

```
String dump of section '.shstrtab':
```

```
[ 1] .symtab
[ 9] .strtab
[11] .shstrtab
[1b] .rela.text
[26] .data
[2c] .bss
[31] .rela.data.rel.local
[46] .rodata
[4e] .rela.debug_info
[5f] .debug_abbrev
[6d] .rela.debug_aranges
[81] .rela.debug_line
[92] .debug_str
[9d] .comment
[a6] .note.GNU-stack
[b6] .rela.eh_frame
```


显示某个节中的内容

Q: 节头表中的名字个数 与 shstrtab 中的不一致?

```
.text  
.rela.text  
.data  
.bss  
.data.rel.local  
.rela.data.rel.local  
.rodata  
.debug_info  
.rela.debug_info  
.debug_abbrev  
.debug_aranges  
.rela.debug_aranges  
.debug_line  
.rela.debug_line  
.debug_str  
.comment  
.note.GNU-stack  
.eh_frame  
.rela.eh_frame  
.symtab  
.strtab  
.shstrtab
```

```
.symtab  
.strtab  
.shstrtab  
.rela.text  
.data  
.bss  
.rela.data.rel.local  
.rodata  
.rela.debug_info  
.debug_abbrev  
.rela.debug_aranges  
.rela.debug_line  
.debug_str  
.comment  
.note.GNU-stack  
.rela.eh_frame
```

.text 就是 .rela.text 中后面的一部分!

.data.rel.local 是 .rela.data.rel.local 的尾部!

.debug_info 是 .rela.debug_info 的尾部!

显示符号表中的内容

□ 显示符号表 Elf64_Sym

#readelf -s test.o

Symbol table '.symtab' contains 22 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000000	0	FILE	LOCAL	DEFAULT	ABS	test.c
2:	0000000000000000	0	SECTION	LOCAL	DEFAULT	1	
3:	0000000000000000	0	SECTION	LOCAL	DEFAULT	3	
4:	0000000000000000	0	SECTION	LOCAL	DEFAULT	4	
5:	0000000000000000	0	SECTION	LOCAL	DEFAULT	5	
6:	0000000000000000	0	SECTION	LOCAL	DEFAULT	6	
7:	0000000000000000	0	SECTION	LOCAL	DEFAULT	8	
8:	0000000000000000	0	SECTION	LOCAL	DEFAULT	9	
9:	0000000000000000	0	SECTION	LOCAL	DEFAULT	11	
10:	0000000000000000	0	SECTION	LOCAL	DEFAULT	13	
11:	0000000000000000	0	SECTION	LOCAL	DEFAULT	15	
12:	0000000000000000	0	SECTION	LOCAL	DEFAULT	16	
13:	0000000000000000	0	SECTION	LOCAL	DEFAULT	14	

说明：开始的一部分符号是 文件名、节名

显示符号表中的内容

Symbol table '.symtab' contains 22 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
14:	000000000000000000	4	OBJECT	GLOBAL	DEFAULT	3	gx
15:	000000000000000004	4	OBJECT	GLOBAL	DEFAULT	COM	gy
16:	000000000000000004	4	OBJECT	GLOBAL	DEFAULT	3	gz
17:	000000000000000000	60	FUNC	GLOBAL	DEFAULT	1	add
18:	00000000000000003c	173	FUNC	GLOBAL	DEFAULT	1	main
19:	000000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	_GLOBAL_OFFSET_T
20:	000000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	printf
21:	000000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	__stack_chk_fail

说明：程序中用到的符号

显示符号表中的内容

readelf -s 列名	Elf64_Sym 字段	字段类型	含义说明
Num	—	—	符号表中的条目序号，不是结构体字段
Value	st_value	Elf64_Addr	符号的值：变量地址、函数入口、或 ABS 常量
Size	st_size	uint64_t	符号占用字节数，变量大小 / 函数长度
Type	st_info 低 4 位	uint8_t	符号类型：NOTYPE、OBJECT、FUNC、SECTION、FILE 等
Bind	st_info 高 4 位	uint8_t	绑定属性：LOCAL（本文件可见）、GLOBAL（全局可见）、WEAK
Vis	st_other	uint8_t	可见性：DEFAULT、HIDDEN、PROTECTED 等
Ndx	st_shndx	uint16_t	符号所在节的编号： · 数字 → 对应节号 · ABS → 绝对符号 · UND → 未定义符号
Name	st_name	uint32_t	符号名在字符串表（.strtab）中的偏移

显示符号表 中属于某个节的符号

```
#readelf -s test.o | awk '$7==节号 || NR==行号'
```

awk 是用来 “按列筛选文本” 的神器。

readelf -s 输出的内容是一排一排、一列一列的，
awk 就是帮你按列筛选，只留下你要的行。

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
↑ \$1	↑ \$2	↑ \$3	↑ \$4	↑ \$5	↑ \$6	↑ \$7	↑ \$8

NR = Number of Record = 行号，

就是保留 **readelf -s** 输出的指定行号的内容

```
#readelf -s test.o | awk '$7==3 || NR==3'
```

保留节号为3的行， 以及 第3行

```
#readelf -s test.o | awk '$7=="UND"'      显示未定义符号
```

显示代码节重定位节中的内容

- 显示.text 的重定位节内容 Elf64_Rela;
 readelf -R .rela.text test.o

```
root@LAPTOP-CJLSTBTI:/home/chapter4# readelf -R .rela.text test.o

Hex dump of section '.rela.text':
0x00000000 17000000 00000000 02000000 0e000000 .....
0x00000010 fcffffff ffffffff 1d000000 00000000 .....
0x00000020 02000000 0f000000 fcffffff ffffffff .....
0x00000030 28000000 00000000 02000000 0f000000 (.
0x00000040 fcffffff ffffffff 2e000000 00000000 .....
0x00000050 02000000 10000000 fcffffff ffffffff .....
0x00000060 7c000000 00000000 02000000 11000000 |.....
```

显示代码节重定位节中的内容

□ 显示重定位节

readelf -r test.o (各重定位节都显示)

readelf -r 列名	Elf64_Rela 字段	字段类型	含义说明
OFFSET	r_offset	Elf64_Addr	需要被重定位修改的位置 即：在 .text/.data 等节内的 字节偏移
INFO	r_info	Elf64_Xword	复合字段： · 低 32 位：重定位类型 (如 R_X86_64_64、 R_X86_64_PC32) · 高 32 位：符号表索引 (对应 readelf -s 里的符号 序号)
TYPE	从 r_info 解析	—	重定位计算方式 (由链接 器 / 加载器解释)
SYMBOL	从 r_info 解析	—	要引用的符号名 (通过符 号表索引找到)
ADDEND	r_addend	Elf64_Sxword	有符号常量加数 最终值 = 符号地址 + r_addend

显示代码节重定位节中的内容

□ 显示 .rel.text 重定位节

```
readelf -r test.o | grep -A 20 "rel.text"
```

-A 20 显示 20行

学会求助

你想做什么操作？ 你想达到什么效果？

不会做，问豆包等

不理解豆包的回答，再问豆包

如果按照回答操作，未达到相应的效果，
可给出更详细的信息，再次问豆包